Discrete Mathematics Coursework Assignment

Semester 1

Table of Contents

Introduction..... 2
Coursework Tasks..... 4
Task 1: Data Type Identification..... 4
Task 2: Central Tendency Analysis..... 6
Task 3: Variability and Distribution..... 10
Task 4: Trend Analysis..... 13
Task 5: Solving Systems of Equations..... 17
Bibliography..... 19

Introduction

“Cities account for over 70% of global CO_2 emissions, most of which come from industrial and motorized transport systems that use huge quantities of fossil fuels and rely on far-flung infrastructure constructed with carbon-intensive materials.” – World Bank Blogs

This should not appear as a surprise - that cities and urban areas are generally regarded as one of the most polluted areas on the planet. From thousands to millions of residents living in tightly packed areas, pollution can be disguised in all types of shapes and forms, things that even you sometimes don’t pay attention to as you can’t see it with the naked eye; examples as covered in this coursework assignment include the following: Increasing traffic congestion, CO2 emissions from industrial plants powering these areas as well as exhaust fumes, and even sound pollution.

All these factors affect each resident’s health, especially those with health conditions such as asthma and must be addressed correctly and effectively, with solutions ranging from city to city, going even further to sections of the city, from downtown to uptown. Henceforth, this coursework is extremely significant for this exact reason, taking on the role of a Junior Data Analyst to work on a life like model on a dataset with uniquely collected data for each student.

More precisely, the data collected from the Smart Synthetic City includes the following:

- Traffic Flow – Unsigned Integer Value Measured in Vehicles Per Hour
- PM25 (or PM2.5) – Unsigned Integer Value Measured in Micrograms/Cubic Meter of Air
- Noise – Unsigned Integer Value Measured in Decibels
- Temperature – Signed Float/Real Value Measured in Degrees Celsius
  (Listed as signed datatype value – however, all values positive in dataset provided)
- Day Type – Qualitative Data of String Value (Weekday or Weekend)
- Region – Qualitative Data of String Value Too (North, East, South, West)

The aim of this coursework assignment is to perform statistical and mathematical analysis of a uniquely identifying, synthetic smart city’s environmental dataset containing data of the type shown above. To reiterate, the significance of this assignment’s scenario should not be understated; it will provide practical experience applicable to the real-life world through making conclusions from the data collected through synthetic data collection practices such as utilizing sensors and analysed from the CSV dataset. Due to the rising problems of environmental issues, this coursework is on the right timeline as these types of problems are realistic and vary based on their respective environments.

The tasks in this assignment:

- Data Type Identification – Requires the identification of type of data being dealt with, classifying them into discrete/continuous and qualitative/quantitative, visually representing this information in a table.
- Central Tendency Analysis – Calculate mean, mode, median of 3 variables, compare and interpret what they mean, spot outliers.
- Variability and Distribution – Describe the spread of data, identifying any repeated patterns using a histogram created in Python, as well as finding the range of 2 different variables.
- Trend Analysis - Describe the spread of data, identifying any repeated patterns using a line plot and bar chart comparing filtered data created in Python
- Solving Systems of Equations – Finding the mean of 3 different variables, 2 are used to predict the PM2.5 mean value, the last one is used to verify if the model is accurate by finding the true PM2.5 mean value.
- Optimization Problem – Find total combinations possible, maximum PM2.5 Level reduction combination and value, then create a bar chart to visually show which is the optimal solution.

Statistical and mathematical analysis will be integrated through the knowledge gained from lectures, tutorials and individual research. Some of the solutions will include Python implementations that can read, manipulate, process and represent the dataset, clearly showing the steps taken to solve each problem with each task having a 50-word conclusion limit to be concise. Python is a very useful tool which can represent them visually using graphs and other tools available in the matplotlib library, or other applicable representations. The tasks assigned are capped at a 50-word limit each; promoting concise conclusions based on the task undertaken and achieved. The tasks themselves range in difficulty, getting progressively more difficult working your way down from the list. For example, the first task involves identifying data types varying all the way to using prediction models, analysing trends and optimization techniques.

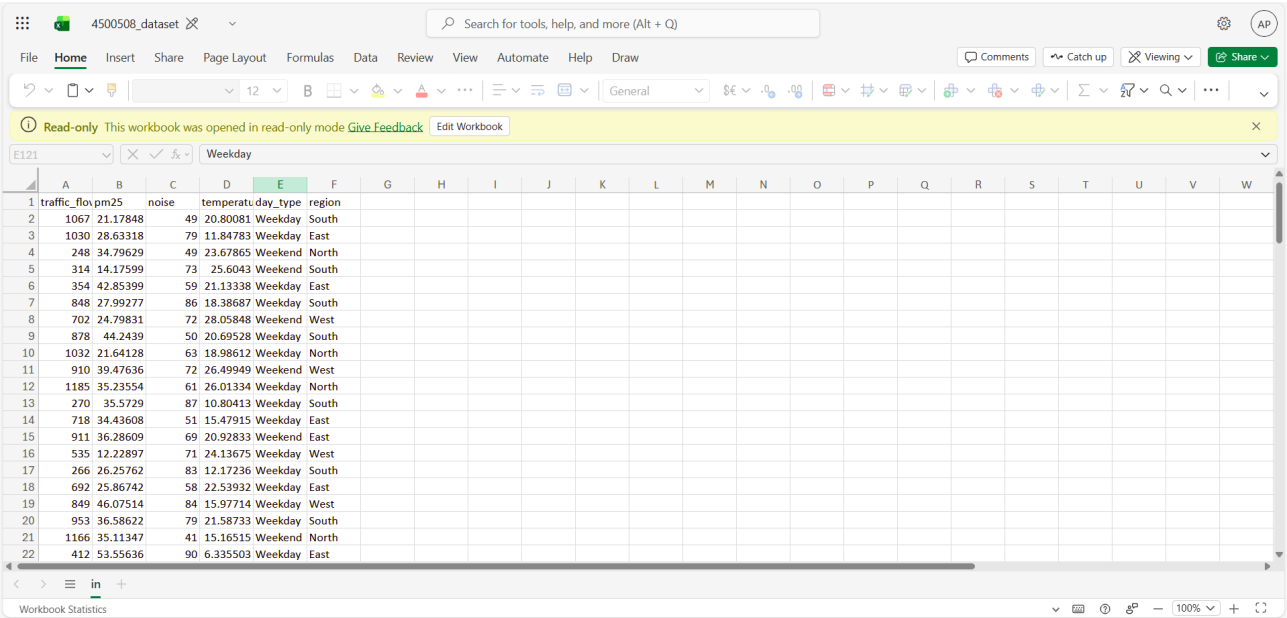
Lastly, another important objective to be achieved and to not go over, is that this report must be presented in a formal, professional manner and well documented manner with a consultancy-style analytical report form, contributing to the overall success criteria and marks gained from this coursework.

Word Limit Followed (1966 words, including references)

Coursework Tasks

Task 1: Data Type Identification

dataset.csv (excel) file:



	A	B	C	D	E	F
1	traffic_flov	pm25	noise	temperatu	day_type	region
2	1067	21.17848	49	20.80081	Weekday	South
3	1030	28.63318	79	11.84783	Weekday	East
4	248	34.79629	49	23.67865	Weekend	North
5	314	14.17599	73	25.6043	Weekend	South
6	354	42.85399	59	21.13338	Weekday	East
7	848	27.99277	86	18.38687	Weekday	South
8	702	24.79831	72	28.05848	Weekend	West
9	878	44.2439	50	20.69528	Weekday	South
10	1032	21.64128	63	18.98612	Weekday	North
11	910	39.47636	72	26.49949	Weekend	West
12	1185	35.23554	61	26.01334	Weekday	North
13	270	35.5729	87	10.80413	Weekday	South
14	718	34.43608	51	15.47915	Weekday	East
15	911	36.28609	69	20.92833	Weekend	East
16	535	12.22897	71	24.13675	Weekday	West
17	266	26.25762	83	12.17236	Weekday	South
18	692	25.86742	58	22.53932	Weekday	East
19	849	46.07514	84	15.97714	Weekday	West
20	953	36.58622	79	21.58733	Weekday	South
21	1166	35.11347	41	15.16515	Weekend	North
22	412	53.55636	90	6.335503	Weekday	East

Expanded:

	A	B	C	D	E	F
1	traffic_flov	pm25	noise	temperatu	day_type	region
2	1067	21.17848	49	20.80081	Weekday	South
3	1030	28.63318	79	11.84783	Weekday	East
4	248	34.79629	49	23.67865	Weekend	North
5	314	14.17599	73	25.6043	Weekend	South
6	354	42.85399	59	21.13338	Weekday	East
7	848	27.99277	86	18.38687	Weekday	South
8	702	24.79831	72	28.05848	Weekend	West
9	878	44.2439	50	20.69528	Weekday	South
10	1032	21.64128	63	18.98612	Weekday	North

There are 6 rows: **traffic_flow, pm25, noise, temperature, day_type, region**

Qualitative Data – are numeric/measurable & Quantitative Data – are categories or labels

- The following are qualitative: **traffic_flow, pm25, noise, temperature**
- The following are quantitative: **day_type, region**

Discrete Data – are countable or take in specific values & Continuous Data – are values within a given range

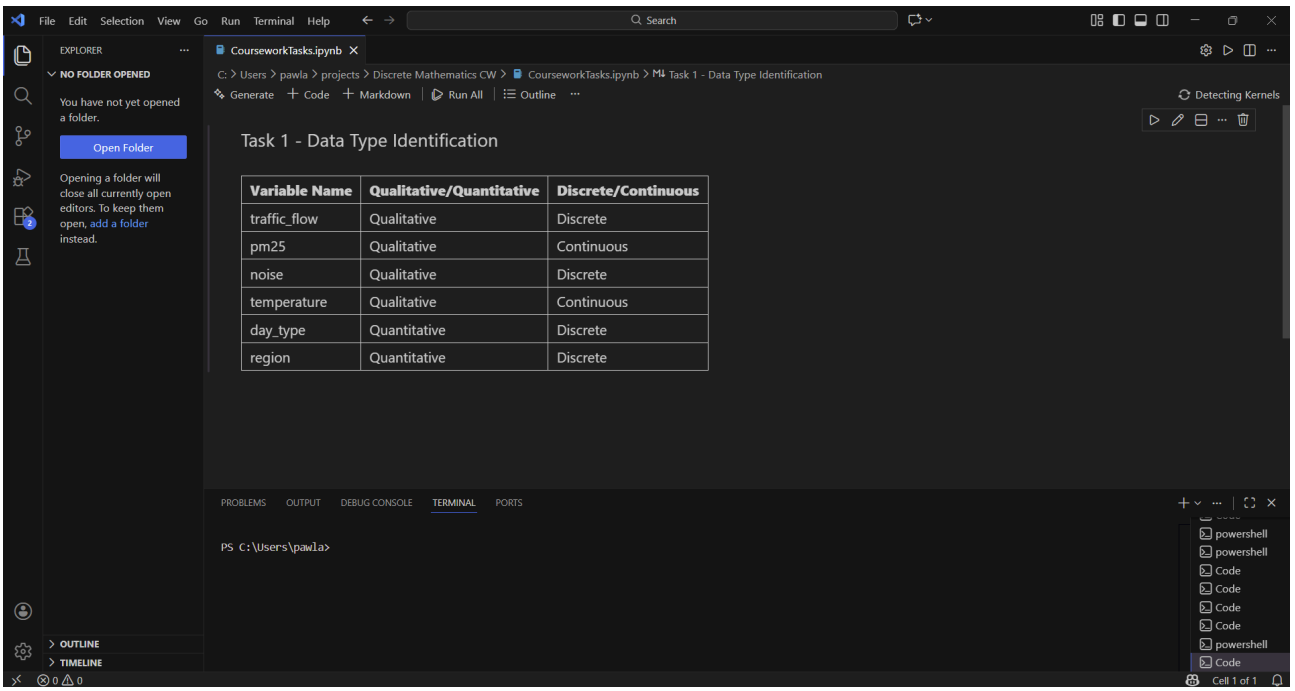
- The following are discrete: **traffic_flow, noise, day_type, region**
- The following are continuous: **pm25, temperature**

The variables **day_type** and **region** can be debated whether they are discrete data types as they take in a limited number of values, that are non-numerical and more categorical. However, the task instructs to choose between the two given, and they are much more appropriately classified as discrete than continuous for this reason.

Simple Summary Table:

Variable Name	Qualitative/Quantitative	Discrete/Continuous
traffic_flow	Qualitative	Discrete
pm25	Qualitative	Continuous
noise	Qualitative	Discrete
temperature	Qualitative	Continuous
day_type	Quantitative	Discrete
region	Quantitative	Discrete

Completed in Jupyter Notebook (VSCode IDE):



Zoom in:

Task 1 - Data Type Identification

Variable Name	Qualitative/Quantitative	Discrete/Continuous
traffic_flow	Qualitative	Discrete
pm25	Qualitative	Continuous
noise	Qualitative	Discrete
temperature	Qualitative	Continuous
day_type	Quantitative	Discrete
region	Quantitative	Discrete

Markdown Code:

```
#### Task 1 - Data Type Identification
| **Variable Name** | **Qualitative/Quantitative** | **Discrete/Continuous** |
| --- | --- | --- |
| traffic_flow | Qualitative | Discrete |
| pm25 | Qualitative | Continuous |
| noise | Qualitative | Discrete |
| temperature | Qualitative | Continuous |
| day_type | Quantitative | Discrete |
| region | Quantitative | Discrete |
```

```
#### Task 1 - Data Type Identification
| **Variable Name** | **Qualitative/Quantitative** | **Discrete/Continuous** |
| --- | --- | --- |
| traffic_flow | Qualitative | Discrete |
| pm25 | Qualitative | Continuous |
| noise | Qualitative | Discrete |
| temperature | Qualitative | Continuous |
| day_type | Quantitative | Discrete |
| region | Quantitative | Discrete |
```

Explanation:

- #### Creates a heading, the more #'s, the smaller the text size
- ** ** any text within these double starts is bold
- | allows creation of table, separates the columns

Task 2: Central Tendency Analysis

File Explorer Folder (Setup):

Name	Date modified	Type	Size
4500508_dataset	26/11/2025 13:21	Comma Separated...	8 KB
CourseworkTasks	17/12/2025 23:40	Jupyter Source File	1 KB
DMA-CW-Task2-CentralTendencyAnalysis...	17/12/2025 23:44	PY File	0 KB

These must be in the same folder for the Python code library Pandas to be able to read from the .csv file

Full Python Code + Terminal

CourseworkTasks.ipynb | DMA-CW-Task2-CentralTendencyAnalysis.py | 4500508_dataset.csv

DMA-CW-Task2-CentralTendencyAnalysis.py

```python
#Central Tendency Analysis
import pandas as pd #library for data sets - allow for use of functions such as .readcsv()

database = pd.read_csv("4500508_dataset.csv") #accesses csv file

#re-usable function that finds mean, median & mode
def calculations(variable):
    mean = database[variable].mean() #calculates mean
    median = database[variable].median() #calculates median
    mode = database[variable].mode() #calculates mode
    return f"Variable:{variable}\nMean:{mean}\nMedian:{median}\nMode:{mode}\n" #returns all values

print(calculations("traffic_flow")) #passes string/column "traffic_flow" through function calculations, variable = "traffic_flow"
print(calculations("pm25")) #passes string/column "pm25" through function calculations, variable = "pm25"
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code

```
PS C:\Users\pawla\projects\Discrete Mathematics CW> python -u "c:\Users\pawla\projects\Discrete Mathematics CW\DMA-CW-Task2-CentralTendencyAnalysis.py"
Variable:traffic_flow
Mean:748.0152671755725
Median:770.0
Mode:0    1080
Name: traffic_flow, dtype: int64

Variable:pm25
Mean:34.6238546240121
Median:35.23553677511291
Mode:0       5.000000
1       8.331602
2       8.754245
3      12.228970
4      14.175993
          ...
126    51.638880
127    52.813831
128    53.556357
129    57.841870
130    60.260830
Name: pm25, Length: 131, dtype: float64
```

Zoom In:

DMA-CW-Task2-CentralTendencyAnalysis.py

```python
#Central Tendency Analysis
import pandas as pd #library for data sets - allow for use of functions such as .readcsv()

database = pd.read_csv("4500508_dataset.csv") #accesses csv file

#re-usable function that finds mean, median & mode
def calculations(variable):
    mean = database[variable].mean() #calculates mean
    median = database[variable].median() #calculates median
    mode = database[variable].mode() #calculates mode
    return f"Variable:{variable}\nMean:{mean}\nMedian:{median}\nMode:{mode}\n" #returns all values

print(calculations("traffic_flow")) #passes string/column "traffic_flow" through function calculations, variable = "traffic_flow"
print(calculations("pm25")) #passes string/column "pm25" through function calculations, variable = "pm25"
```

```
Variable:traffic_flow
Mean:748.0152671755725
Median:770.0
Mode:0    1080
```

```
Variable:pm25
Mean:34.6238546240121
Median:35.23553677511291
Mode:0       5.000000
```

Table:

Variable	Mean	Median	Mode
traffic_flow	748.02 (2dp)	770	1080
pm25	34.62 (2dp)	35.24 (2dp)	5.0*

The traffic flow mean should be a whole number; therefore it is appropriate to round it down, you can't have 0.02 of a car. *Finally, the pm25 mode should be a null value, as there isn't a pair of values/repeats. In the csv file, there is only one value of 5.

Completed in Jupyter Notebook (VSCode IDE + Markdown):

Task 2 - Data Type Identification

Variable Name	Mean	Median	Mode
traffic_flow	748.02 (2dp)	770	1080
pm25	34.62 (2dp)	35.24 (2dp)	5.0

```
#### Task 2 - Data Type Identification
| **Variable Name** | **Mean** | **Median** | **Mode** |
| --- | --- | --- | --- |
| traffic_flow | 748.02 (2dp) | 770 | 1080 |
| pm25 | 34.62 (2dp) | 35.24 (2dp) | 5.0 |
```

Markdown Code:

```
#### Task 2 - Data Type Identification
| **Variable Name** | **Mean** | **Median** | **Mode** |
| --- | --- | --- | --- |
| traffic_flow | 748.02 (2dp) | 770 | 1080 |
| pm25 | 34.62 (2dp) | 35.24 (2dp) | 5.0 |
```

The results show the mean (average), median (middle value arranged from lowest to highest – outliers excluded), mode (most common value).

Data Analysis (Interpretation):

748 cars per hour is the average daily traffic volume around the city & 34.62 Micrograms/Cubic Meter of air is the average daily pollution level.

Half of the recorded traffic volume is below 770 and half above, median is greater than mean so there are some days where traffic volume is very low. Half of the pollution level is below 35.24 and half above, very close to mean hinting stable pollution levels.

Most common traffic volume is 1080 vehicles per hour which is much greater than the mean, further suggesting there are days of high and low traffic volumes whereas pollution level doesn't have a mode.

Task 3: Variability and Distribution

Calculating Range:

CourseworkTasks.ipynb | DMA-CW-Task2-CentralTendencyAnalysis.py | DMA-CW-Task3-VariabilityAndDistribution.py | 4500508_dataset.csv

DMA-CW-Task3-VariabilityAndDistribution.py

```python
#Central Tendency Analysis
import pandas as pd #library for data sets - allow for use of functions such as .readcsv()

database = pd.read_csv("4500508_dataset.csv") #accesses csv file

#re-usable function that finds range
def calculations(variable):
    maximum = database[variable].max() #stores max value in variable maximum
    minimum = database[variable].min() #stores minimum value in variable minimum
    variable_range = maximum - minimum #calculates range
    return f"Variable:{variable}\nRange:{variable_range}\n" #returns all values

print(calculations("traffic_flow")) #passes string/column "traffic_flow" through function calculations, variable = "traffic_flow"
print(calculations("noise")) #passes string/column "noise" through function calculations, variable = "noise"
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
PS C:\Users\pawla\projects\Discrete Mathematics CW> python -u "c:\Users\pawla\projects\Discrete Mathematics CW\DMA-CW-Task3-VariabilityAndDistribution.py"
Variable:traffic_flow
Range:996

Variable:noise
Range:54
```

Zoom In:

DMA-CW-Task3-VariabilityAndDistribution.py

```python
#Central Tendency Analysis
import pandas as pd #library for data sets - allow for use of functions such as .readcsv()

database = pd.read_csv("4500508_dataset.csv") #accesses csv file

#re-usable function that finds range
def calculations(variable):
    maximum = database[variable].max() #stores max value in variable maximum
    minimum = database[variable].min() #stores minimum value in variable minimum
    variable_range = maximum - minimum #calculates range
    return f"Variable:{variable}\nRange:{variable_range}\n" #returns all values

print(calculations("traffic_flow")) #passes string/column "traffic_flow" through function calculations, variable = "traffic_flow"
print(calculations("noise")) #passes string/column "noise" through function calculations, variable = "noise"
```

```
Variable:traffic_flow
Range:996
```

```
Variable:noise
Range:54
```

To verify these values, I used basic excel and converted the excel to a .xlsx file:

traffic_flo	pm25	noise	temperatu	day_type	region
1067	21.17848	49	20.80081	Weekday	South
1030	28.63318	79	11.84783	Weekday	East
248	34.79629	49	23.67865	Weekend	North
314	14.17599	73	25.6043	Weekend	South
354	42.85399	59	21.13338	Weekday	East
848	27.99277	86	18.38687	Weekday	South
702	24.79831	72	28.05848	Weekend	West
878	44.2439	50	20.69528	Weekday	South
1032	21.64128	63	18.98612	Weekday	North
910	39.47636	72	26.49949	Weekend	West
1185	35.23554	61	26.01334	Weekday	North
270	35.5729	87	10.80413	Weekday	South
718	34.43608	51	15.47915	Weekday	East
911	36.28609	69	20.92833	Weekend	East
535	12.22897	71	24.13675	Weekday	West
266	26.25762	83	12.17236	Weekday	South
692	25.86742	58	22.53932	Weekday	East
849	46.07514	84	15.97714	Weekday	West
953	36.58622	79	21.58733	Weekday	South
1166	35.11347	41	15.16515	Weekend	North
412	53.55636	90	6.335503	Weekday	East
380	14.75531	53	9.144196	Weekday	South
965	49.14484	94	12.3696	Weekday	West

=MAX(A2:A132)	
202	Min TF
996	TF Range
94	Max Noise
40	Min Noise
54	Noise Range

I used the =MAX(), =MIN(), = functions in excel, which match what my python code returned.

Noise Level Histogram (matplotlib):

```
import matplotlib.pyplot as plt #library to plot graphs - plt.command
```

```
#function used to draw graph - histogram
def draw(variable):
    plt.figure(figsize=(10, 6)) #defines height and width
    database[variable].dropna().plot(kind="hist", bins=11, edgecolor="black") #bins means intervals, plots histogram
    plt.title(f"Histogram of {variable}") #graph title
    plt.xlabel(variable) #label x axis with variable name
    plt.ylabel("Frequency") #label y axis with Frequency
    plt.show() #displays graph

draw("noise") #calls function
```

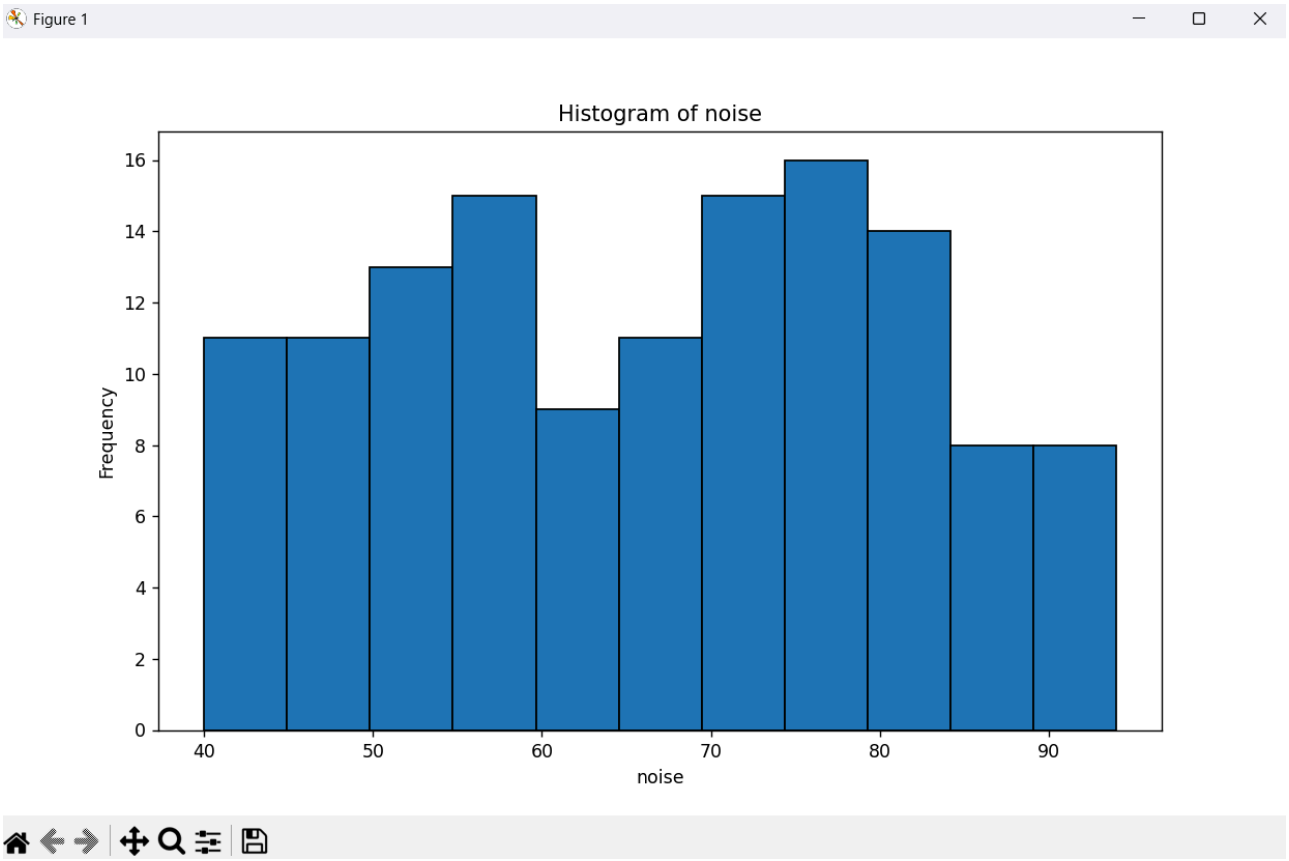
Both of the screenshots on the next page represent the same histogram: Noise Level

The second one has double the bins/intervals

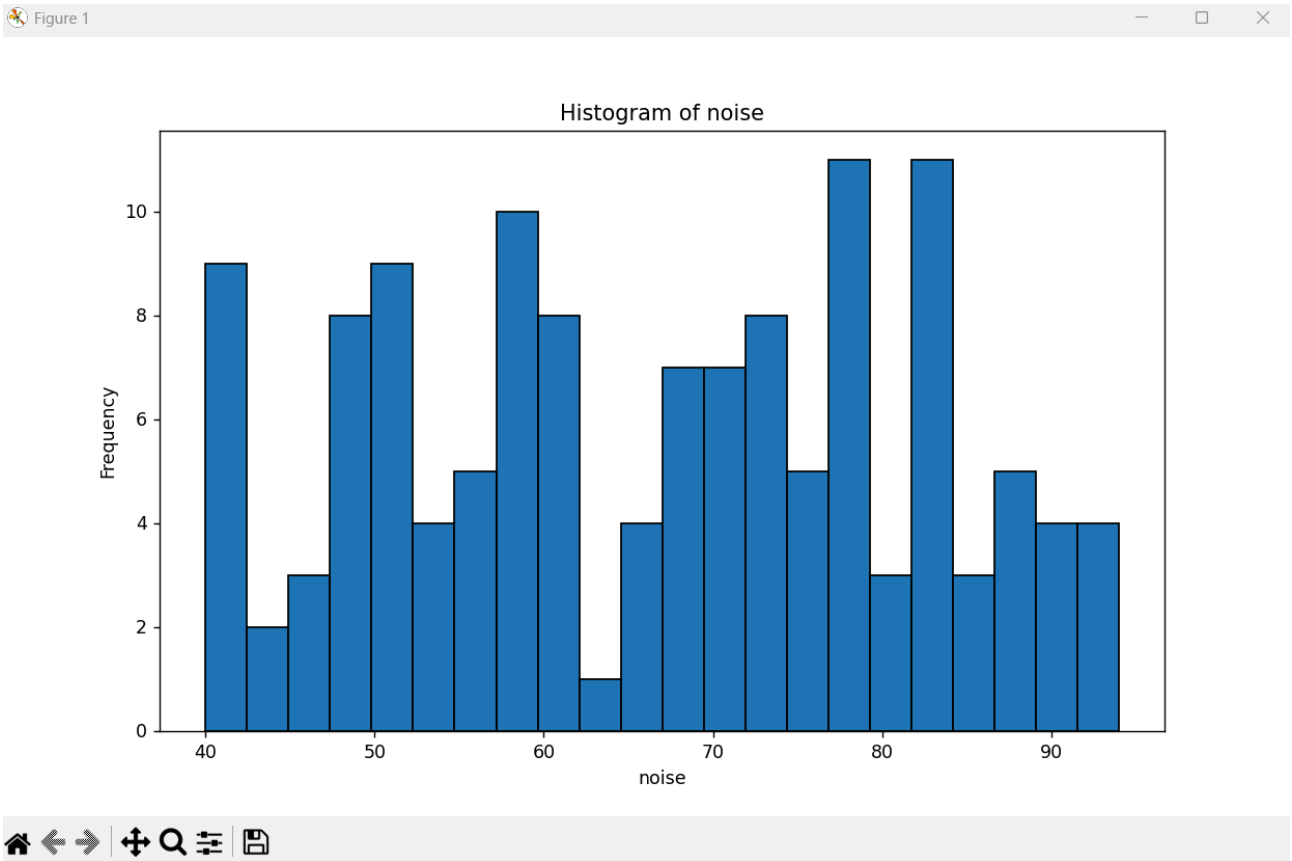
25/11 = roughly 4Db per interval

45/22 = roughly 2Db per interval

Histogram 1 (11 bins/intervals):



Histogram 2 (22 bins/intervals)



Interpretation of Ranges:

Very high range in traffic volume, due to daytime and night-time activity, noise levels much lower range comparatively to traffic volume, more consistent.

Interpretation of Histogram:

This shows that the noise levels are highly variable most likely due to daytime and nighttime activity. Going more in depth, nighttime on weekdays is much less likely to be higher than on weekends due to people going out as an example. Most noise levels are before approximately 85Db as really sounds are less common. The two least common noise levels are between 62-64Db and 42-44Db.

Task 4: Trend Analysis

Calculating average Traffic Flow for each Day Type:

```python
#Trend Analysis
import pandas as pd #library for data sets - allow for use of functions such as .readcsv()
import matplotlib.pyplot as plt #library to plot graphs - plt.command

database = pd.read_csv("4500508_dataset.csv") #accesses csv file

#re-usable function that finds range
def calculations(variable):
    weekday = database[database["day_type"]=="Weekday"] #filters database by values where column day_type is Weekday
    weekend = database[database["day_type"]=="Weekend"] #filters database by values where column day_type is Weekend
    weekdayValues = weekday[variable].mean() #calculates mean of traffic flow where day_type is Weekday
    weekendValues = weekend[variable].mean() #calculates mean of traffic flow where day_type is Weekend
    return f"Variable:{variable}\nWeekday Mean:{weekdayValues}\nWeekend Mean:{weekendValues}\n" #returns all values

print(calculations("traffic_flow")) #passes string/column "traffic_flow" through function calculations, variable = "traffic_flow"
```

```
PS C:\Users\pawla\projects\Discrete Mathematics CW> python -u "c:\Users\pawla\projects\Discrete Mathematics CW\DMA-CW-Task4-TrendAnalysis.py"
Variable:traffic_flow
Weekday Mean:723.7777777777778
Weekend Mean:801.219512195122
```

Zoom In (Python Code):


```python
#Trend Analysis
import pandas as pd #library for data sets - allow for use of functions such as .readcsv()
import matplotlib.pyplot as plt #library to plot graphs - plt.command

database = pd.read_csv("4500508_dataset.csv") #accesses csv file

#re-usable function that finds range
def calculations(variable):
    weekday = database[database["day_type"]=="Weekday"] #filters database by values where column day_type is Weekday
    weekend = database[database["day_type"]=="Weekend"] #filters database by values where column day_type is Weekend
    weekdayValues = weekday[variable].mean() #calculates mean of traffic flow where day_type is Weekday
    weekendValues = weekend[variable].mean() #calculates mean of traffic flow where day_type is Weekend
    return f"Variable:{variable}\nWeekday Mean:{weekdayValues}\nWeekend Mean:{weekendValues}\n" #returns all values

print(calculations("traffic_flow")) #passes string/column "traffic_flow" through function calculations, variable = "traffic_flow"
```


Terminal

```
Variable:traffic_flow
Weekday Mean:723.7777777777778
Weekend Mean:801.219512195122
```

Verifying Solution with Simple Excel:

traffic_flo	pm25	noise	temperatu	day_type	region
1067	21.17848	49	20.80081	Weekday	South
1030	28.63318	79	11.84783	Weekday	East
248	34.79629	49	23.67865	Weekend	North
314	14.17599	73	25.6043	Weekend	South
354	42.85399	59	21.13338	Weekday	East
848	27.99277	86	18.38687	Weekday	South
702	24.79831	72	28.05848	Weekend	West
878	44.2439	50	20.69528	Weekday	South
1032	21.64128	63	18.98612	Weekday	North
910	39.47636	72	26.49949	Weekend	West
1185	35.23554	61	26.01334	Weekday	North
270	35.5729	87	10.80413	Weekday	South
718	34.43608	51	15.47915	Weekday	East

1198	Max TF
202	Min TF
996	TF Range
94	Max Noise
40	Min Noise
54	Noise Range
Weekday Mean	723.7778
Weekend Mean	801.2195

```
=AVERAGEIF(E2:E132,"Weekday",A2:A132)
```

723.7778

```
=AVERAGEIF(E2:E132,"Weekend",A2:A132)
```

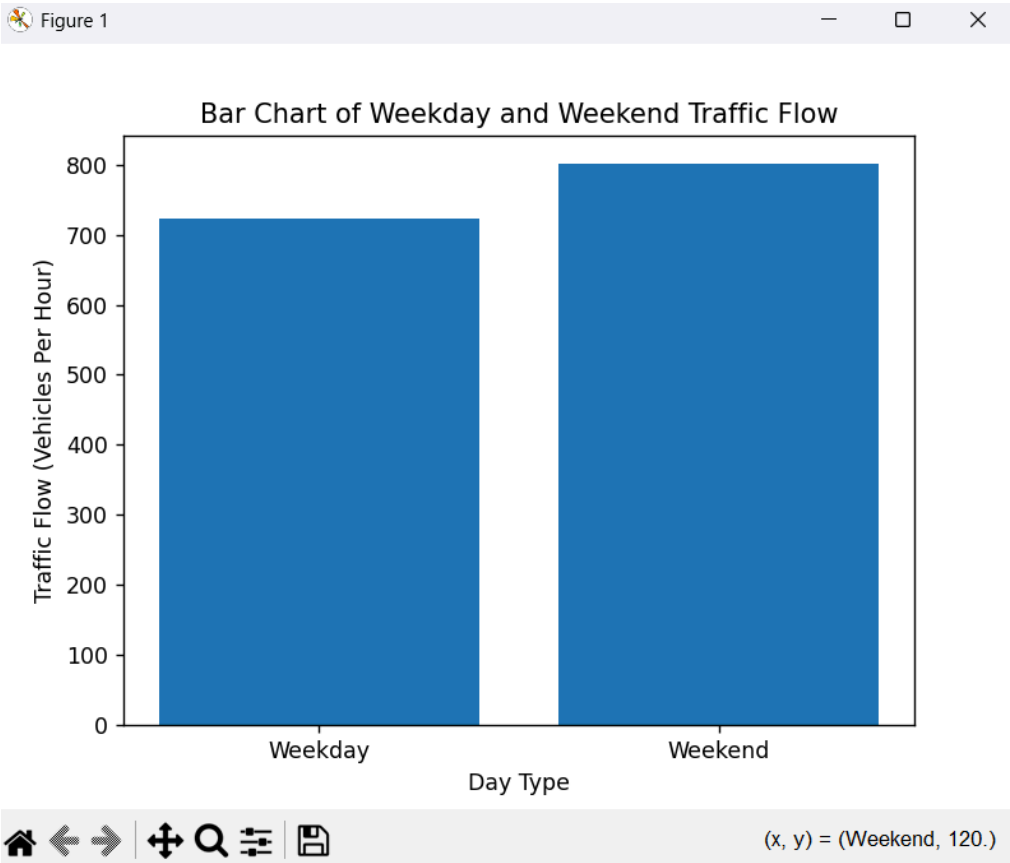
801.2195

Creating Bar Chart:

```
def calculations(variable):
    global weekdayValues,weekendValues #defined as globals so I can use them in draw function
```

```
#function used to draw graph - barchart
def draw():
    fig, figure = plt.subplots() #creates figure
    barX = ['Weekday','Weekend'] #x axis data plots
    barY = [weekdayValues,weekendValues] #y axis data plots
    figure.bar(barX,barY) #creates bar chart .bar(x,y)
    plt.title("Bar Chart of Weekday and Weekend Traffic Flow") #graph title
    plt.xlabel("Day Type") #label x axis
    plt.ylabel("Traffic Flow (Vehicles Per Hour)") #label y axis
    plt.show() #displays graph

draw() #calls function
```



This shows that on weekdays, there are 70 less vehicles per hour, a lower traffic volume than on weekends.

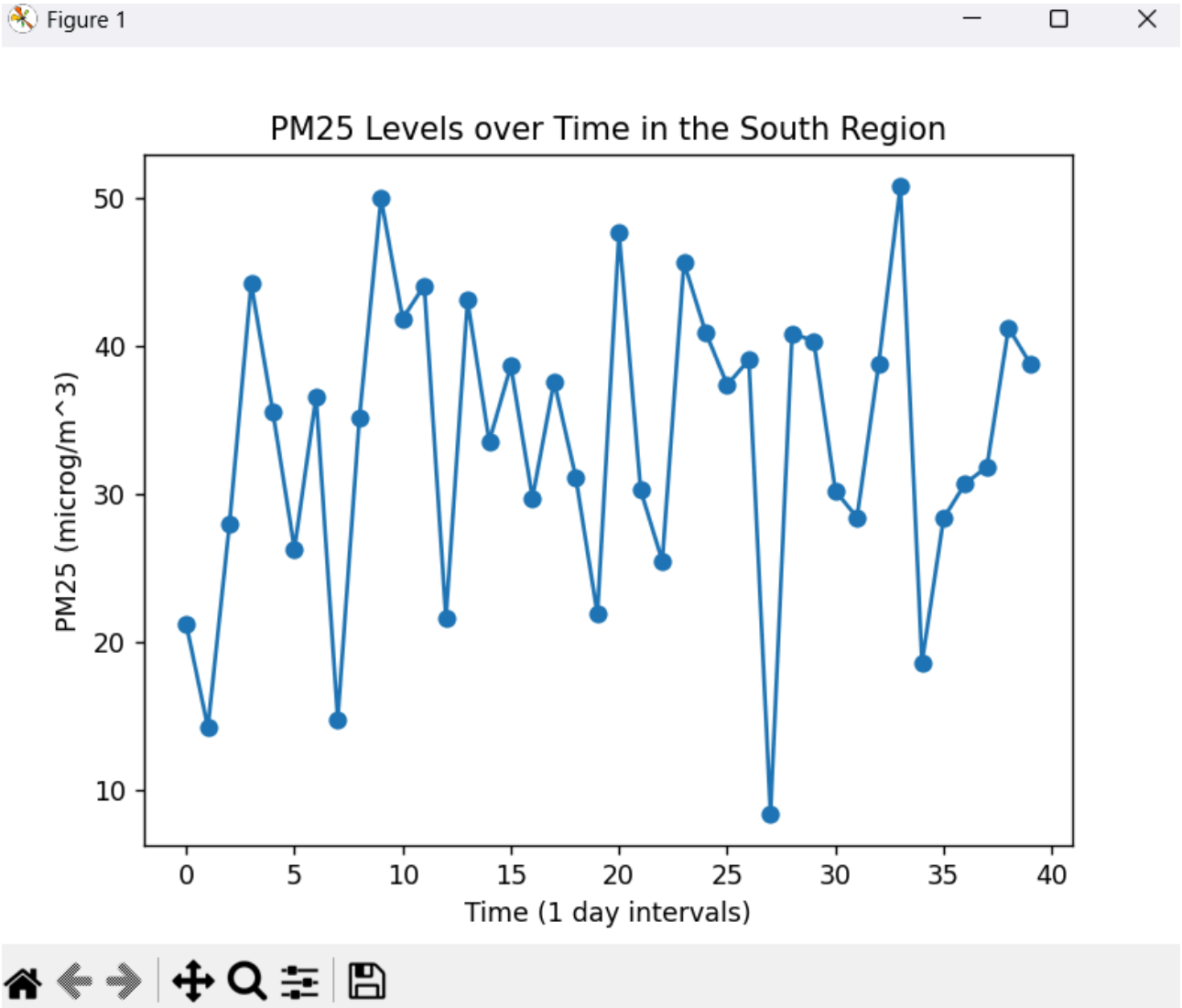
Line Plot (PM25 in South Region):

```
=COUNTIF(F2:F132,"South")
```

Returns 40 values

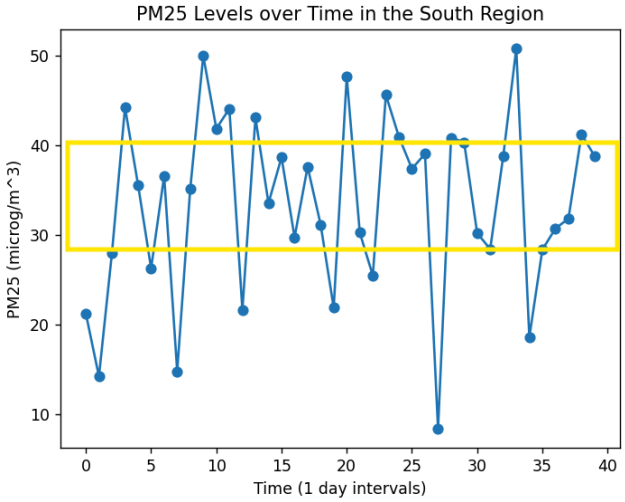
```
#Line plot
region = database[database["region"]=="South"].reset_index(drop=True) #filters database by values where region is South

plt.plot(region.index, region["pm25"], marker="o") #plots line graph .plot(x,y)
plt.title("PM25 Levels over Time in the South Region")
plt.xlabel("Time (1 day intervals)")
plt.ylabel("PM25 (micrograms/m^3)")
plt.show() #displays graph
```



Observation

Most of the recorded values clutter between the following region:



It also shows that there isn't a distinct pattern, fluctuating a lot. It goes up (1-2 times) □ down (1-2) except towards the end where it goes up 4 times in a row from day 35-38 (technically day 36-39 as python counts from 0)

Most studies indicate PM2. 5 at or below 12 μg/m^3 is considered healthy with little to no risk from exposure. If the level goes to or above 35 μg/m^3 during a 24-hour period, the air is considered unhealthy and can cause issues for people with existing breathing issues such as asthma.

Indoor Air Hygiene Institute
https://www.indoorairhygiene.org › pm2-5-explained

PM2.5 Explained - Indoor Air Hygiene Institute

The air quality itself in the Smart City's South region is quite unhealthy, looking at the graph approximately 40-50% values are above or equal to 35 micrograms per cubic meter.

Task 5: Solving Systems of Equations

Predicted PM2.5 Values using Mathematical Model P=0.5T + 0.3N

DMA-CW-Task2-CentralTendencyAnalysis.py | DMA-CW-Task3-VariabilityAndDistribution.py | DMA-CW-Task4-TrendAnalysis.py | DMA-CW-Task5-Solving Systems of Equations.py

DMA-CW-Task5-Solving Systems of Equations.py

```python
#Solving Systems of Equations (P=0.5T+0.3N)
import pandas as pd #library for data sets - allow for use of functions such as .readcsv()
import matplotlib.pyplot as plt #library to plot graphs - plt.command

database = pd.read_csv("4500508_dataset.csv") #accesses csv file

#re-usable function that finds mean
def calculations(variable):
    mean = database[variable].mean() #calculates mean of the variable
    return f"Variable:{variable}\nMean:{mean}" #returns all values

print(calculations("traffic_flow")) #passes string/column "traffic_flow" through function calculations, variable = "traffic_flow"
print(calculations("noise"))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\pawla\projects\Discrete Mathematics CW> python -u "c:\Users\pawla\projects\Discrete Mathematics CW\DMA-CW-Task5-Solving Systems of Equations.py"
Variable:traffic_flow
Mean:748.0152671755725
Variable:noise
Mean:66.3969465648855
```

<u>Zoom In:</u>

```python
#Solving Systems of Equations (P=0.5T+0.3N)
import pandas as pd #library for data sets - allow for use of functions such as .readcsv()
import matplotlib.pyplot as plt #library to plot graphs - plt.command

database = pd.read_csv("4500508_dataset.csv") #accesses csv file

#re-usable function that finds mean
def calculations(variable):
    mean = database[variable].mean() #calculates mean of the variable
    return f"Variable:{variable}\nMean:{mean}" #returns all values

print(calculations("traffic_flow")) #passes string/column "traffic_flow" through function calculations, variable = "traffic_flow"
print(calculations("noise"))
```

```
Variable:traffic_flow
Mean:748.0152671755725
Variable:noise
Mean:66.3969465648855
```

<u>Substituting these values to find P using our model:</u>

P = 0.5T + 0.3N

P = Predicted PM2.5, T = Mean Traffic Flow, N = Mean Noise Level

393.9267176… = 748.0152672 + 66.39694656

P = 393.92672 (5dp)

<u>Verified using excel:</u>

748.0153	mean traffic flow
66.39695	mean noise level
P = 0.5T + 0.3N	
P =	393.9267

*Already, the value calculated looks unrealistic.

Finding Mean PM2.5:

```
print(calculations("pm25"))
```

```
Variable:pm25
Mean:34.6238546240121
```

Verified in Excel:

Mean PM2.5:	34.62385

=AVERAGE(B2:B132)

Conclusion

The predicted mean PM2.5 level of 393.92672 is completely inaccurate when compared with the actual PM2.5 level of 34.62385. The model P=0.5T+0.3N is therefore invalid to predict the PM2.5 level.

Bibliography

(Harvard Referencing) Editor, (Year), Title, Location: Publisher, LINK:

Susmita Dasgupta, Somik Lall & David Wheeler, (2022), Cutting global carbon emissions: where do cities stand? Online: World Bank Blogs LINK: Cutting global carbon emissions: where do cities stand? (used in introduction)

W3Schools, (N/A), Pandas Read CSV, Online: W3Schools, LINK: Pandas Read CSV (used in task 2)

W3Schools, (N/A), Pandas DataFrame Median() Method, Online: W3Schools, LINK: Pandas DataFrame median() Method (used in task 2)

GeeksForGeeks, (2025), Matplotlib.pyplot.hist() in Python, Online: GeeksForGeeks, LINK: Matplotlib.pyplot.hist() in Python - GeeksforGeeks (used in task 3)

Matplotlib - John Hunter, Darren Dale, Eric Firing, Michael Droettboom and the Matplotlib development team, (2002-2025), matplotlib.pyplot.plot, Online, LINK: matplotlib.pyplot.plot — Matplotlib 3.10.8 documentation (task 4)

Indoor Air Hygeine Institute THE INSIDE ADVANTAGE (2025), PM2.5 Explained, Location: Online, LINK: PM2.5 Explained - Indoor Air Hygiene Institute (task 4)